

Moteur BSP

Détail de toutes les étapes du rendu complet

Ce document explique le fonctionnement complet du moteur de rendu BSP. Du découpage de la carte en ArbreBSP, à l'affichage final d'une frame rendu.

L'idée générale :

On construit l'arbre une seule fois au démarrage, puis à chaque frame on le parcourt pour savoir dans quel ordre dessiner les murs.

Tout repose sur un principe simple, les objets proches occultent les objets lointains, et le BSP nous donne cet ordre simplement.

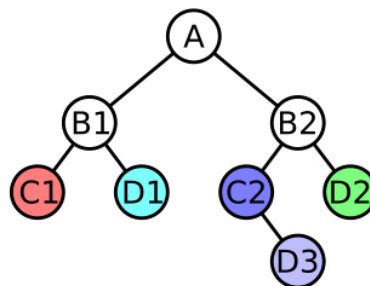
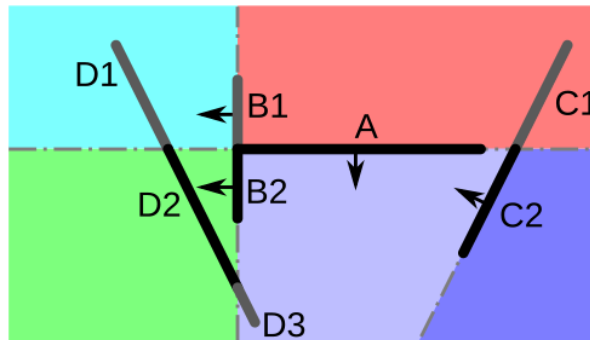
Sommaire:

1. Construction de l'arbre BSP	3
2. Parcours Front-to-Back	4
3. Projection d'un mur en 4 points	5
4. Gestion de l'occlusion, FilledScreen + cropWall	6
FilledScreen.add()	6
WallCalcul.cropWall()	7
5. Insertion des sprites et ordre de dessin	8
6. Rendu final	9
7. Résumé	10

1. Construction de l'arbre BSP

On prend la liste de tous les murs de la map et on les organise en arbre.

Le principe : chaque nœud de l'arbre contient un mur "diviseur" qui coupe l'espace en deux. Les murs d'un côté vont à gauche, les murs de l'autre côté vont à droite. On répète récursivement.



https://fr.wikipedia.org/wiki/Partition_binaire_de_l'espace

```
construireBSP(listeMurs) :  
    si listeMurs est vide -> retourner null  
  
    prendre le PREMIER mur comme diviseur  
    créer un NoeudBSP avec ce mur  
  
    pour chaque autre mur :  
        calculer de quel côté il est (produit croisé : dx*py - dy*px)  
  
        si côté GAUCHE (> epsilon) -> listeMursGauche  
        si côté DROITE (< -epsilon) -> listeMursDroite  
        si COLINÉAIRE (les deux points à 0) -> listeMursGauche (arbitraire)  
        si le mur TRAVERSE le diviseur :  
            calculer le point d'intersection  
            couper le mur en deux  
            placer chaque moitié du bon côté  
  
    noeud.gauche = construireBSP(listeMursGauche) [réursion]  
    noeud.droit = construireBSP(listeMursDroite) [réursion]  
    retourner noeud
```

→ On choisit toujours le premier mur de la liste comme diviseur. Il n'y a pas de méthode magique pour trouver le meilleur mur, dans DOOM, ils exploraient aléatoirement jusqu'à avoir la plus petite découpe de map final.

→ Les murs colinéaires sont mis arbitrairement à gauche pour éviter qu'ils disparaissent ou fasse planter le programme.

2. Parcours Front-to-Back

Une fois l'arbre construit, il faut le parcourir à chaque frame du mur le plus proche au plus loin. C'est un parcours infixe (in-order), mais où à chaque nœud on décide quel côté visiter en premier selon la position du joueur.

L'implémentation est un itérateur paresseux : on appelle getNextWall() en boucle et on peut s'arrêter à tout moment.

La boucle qui a besoin du parcours instantie simplement FrontToBack, puis peut par la suite appeler getNextWall() au besoin jusqu'à avoir fini sont rendu (permet de pouvoir s'arreter plus tôt et permet de séparer la logique du parcours de l'arbre dans une classe à part pour se séparer les tâches)

```
getNextWall() :  
  on maintient une PILE et un pointeur 'courant'  
  au départ : courant = racine  
  
  boucle tant que (courant != null OU pile non vide) :  
  
    si courant != null :  
      pousser courant dans la pile  
      avancer vers le CÔTÉ PROCHE du joueur  
      (= sous-arbre du côté où se trouve le joueur)  
  
    sinon (on est au bout d'une branche) :  
      dépiler un nœud  
      pointer courant vers le CÔTÉ LOIN de ce nœud  
      retourner le mur du nœud dépilé <- c'est le prochain mur  
  
  retourner null quand tout est parcouru
```

→ Le côté proche/loin est déterminé par le même produit croisé qu'à la construction de l'ArbreBSP

→ Ce parcours est totalement interruptible : BSPParcours peut s'arrêter dès que l'écran est plein. Les murs lointains ne sont jamais calculés.

3. Projection d'un mur en 4 points

C'est ici que la 3D devient 2D. Un mur dans le monde est un segment $(x_0, y_0) \rightarrow (x_1, y_1)$. On veut calculer les 4 coins du trapèze qu'il forme à l'écran.

On retourne un objet FourPoints (4 coins dans l'ordre : haut-gauche, bas-gauche, bas-droite, haut-droite). Le Renderer fait ensuite un fillPolygon avec ces 4 points directement -> on peut obtenir une bien meilleure résolution qu'avec le RayCasting qui dépend du nombre de balayage en colonne de l'écran.

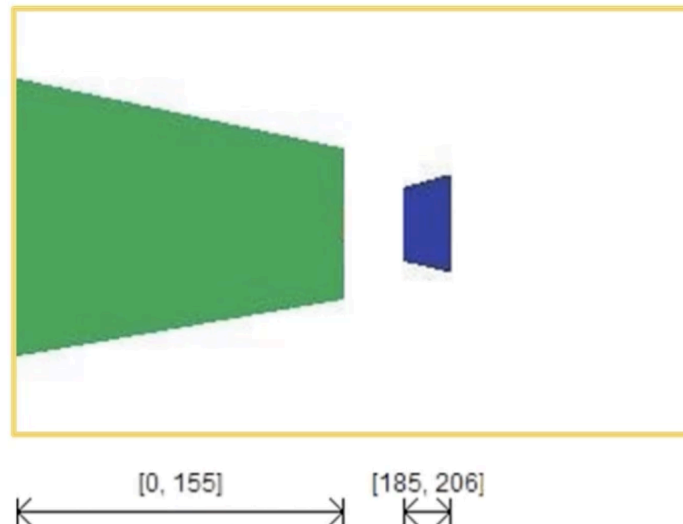
```
getFourPoints(mur, joueurX, joueurY, FOV, angle, largeur, hauteur) :  
  
1. TRANSLATION : ramener le mur dans le repère du joueur  
  wx = mur.x - joueurX  
  wy = mur.y - joueurY  
  
2. ROTATION INVERSE : aligner avec la direction de vue  
  cx = -wx * sin(angle) + wy * cos(angle)    [gauche/droite caméra]  
  cz =  wx * cos(angle) + wy * sin(angle)    [profondeur devant caméra]  
  
3. REJET RAPIDE : si cz0 < 0 ET cz1 < 0 -> le mur est derrière -> null  
  
4. CLIPPING : si un seul point est derrière (cz < epsilon)  
  t = (epsilon - cz0) / (cz1 - cz0)  
  interpoler cx pour garder uniquement la partie visible  
  
5. PROJECTION X (horizontal écran) :  
  scale = (largeur/2) / tan(FOV/2)  
  sx = (cx / cz) * scale + largeur/2  
  
6. PROJECTION Y (hauteur du mur) :  
  haut = (WALL_HEIGHT - PLAYER_HEIGHT) / cz * scale  
  bas  = (-PLAYER_HEIGHT) / cz * scale  
  
7. retourner FourPoints : haut-gauche, bas-gauche, bas-droite, haut-droite
```

→ Plus un point est loin (cz grand), plus sx est ramené vers le centre.

→ Le FourPoints forme un trapèze : si le mur est en biais face au joueur, le côté gauche n'a pas la même hauteur que le côté droit.

4. Gestion de l'occlusion, FilledScreen + cropWall

FilledScreen maintient une liste de segments horizontaux déjà occupés à l'écran. Quand un nouveau mur arrive (en Front-to-Back, donc potentiellement derrière un mur déjà dessiné ou partiellement dessiné), on le découpe pour ne garder que les colonnes encore visibles.

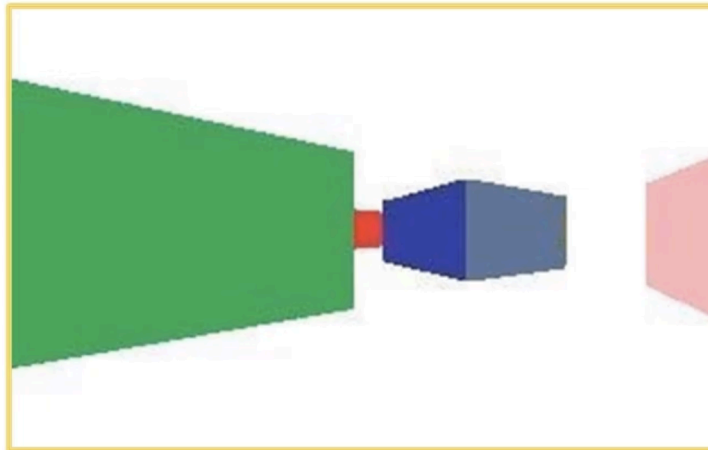


Fonction de FilledScreen, stocker la largeur d'écran rendu

<https://www.youtube.com/watch?v=yTRzfKh4Tg0>

FilledScreen.add()

```
add(FourPoints nouveauMur) :  
    leftX = bord gauche du mur  
    rightX = bord droit du mur  
  
    si [leftX, rightX] entièrement dans un segment déjà rempli :  
        -> retourner null (mur totalement caché, on ignore)  
  
    sinon : appeler cropWall(mur, segmentsDéjàRemplis)  
        -> retourne les morceaux visibles  
  
    pour chaque morceau : ajouter son intervalle à la liste  
        (en fusionnant les segments qui se chevauchent)  
  
    retourner les morceaux visibles
```



Fonction de WallCalcul.cropWall(), réussir à dégrossir les FourPoints si il y a déjà des largeur d'écran rendu. (Le mur rouge a eu ses 4 points occlu car il est derrière des largeurs d'écran déjà remplis)

<https://www.youtube.com/watch?v=yTRzfKh4Tg0>

WallCalcul.cropWall()

```
cropWall(wall, listeZonesOccultées) :
    commencer avec span visible = [wall.x0, wall.x2]  (tout le mur)

    pour chaque zone occultée [zStart, zEnd] :
        pour chaque span visible [sStart, sEnd] :

            zone hors du span          -> garder le span intact
            zone couvre tout le span   -> supprimer le span (caché)
            zone coupe le milieu       -> deux spans : [sStart,zStart] et [zEnd,sEnd]
            zone cache le côté gauche  -> garder [zEnd, sEnd]
            zone cache le côté droit   -> garder [sStart, zStart]

        pour chaque span restant :
            calculer t0 et t1 (ratio de position dans le mur original)
            interpoler les hauteurs haut/bas avec ces ratios
            créer un nouveau FourPoints pour ce sous-segment

    retourner la liste de FourPoints visibles
```

→ L'interpolation des hauteurs est obligatoire : le mur projeté est un trapèze. Si on garde seulement la moitié droite, les y des nouveaux coins doivent être recalculés par interpolation linéaire.

5. Insertion des sprites et ordre de dessin

Les sprites (joueurs, ennemis) ne font pas partie de l'arbre BSP. Ils sont insérés dynamiquement dans la liste ordonnée des murs Front-to-Back, puis tout est retourné pour dessiner en Back-to-Front (algorithme du Peintre).

On retourne tout car c'est le plus simple pour pouvoir occlure les Sprites durant le rendu d'une image. Nos murs en soit n'en ont pas besoin car ils sont dynamiquement occlus durant le parcours. Peu importe l'ordre de rendu des FourPoints ça ne changera rien (sur une Scène sans Sprite, bien évidemment)

```
render() :  
  
    ÉTAPE 1, Collecter les murs visibles en Front-to-Back :  
    orderedItems = []  
    mur = frontToBack.getNextWall()  
    tant que (mur != null ET écran non plein) :  
        projeter le mur -> FourPoints via WallCalcul  
        si FourPoints != null :  
            crop avec FilledScreen -> morceaux visibles  
            ajouter chaque morceau dans orderedItems  
        mur = getNextWall()  
  
    orderedItems est trié PROCHE -> LOIN  
  
    ÉTAPE 2, Insérer les sprites dans la liste :  
    trier les sprites par distance décroissante  
    pour chaque sprite :  
        insertIndex = 0 (par défaut : devant tout)  
        pour chaque mur dans orderedItems :  
            calculer de quel côté du mur est le sprite (produit croisé)  
            calculer de quel côté du mur est le joueur  
            si sprite et joueur sont de CÔTÉS OPPOSÉS :  
                -> le mur est entre eux -> sprite est derrière ce mur  
                -> insertIndex = index_du_mur + 1  
            insérer le sprite à insertIndex dans orderedItems  
  
    ÉTAPE 3, Retourner la liste :  
    Collections.reverse(orderedItems)  
    -> liste maintenant triée LOIN -> PROCHE  
  
    ÉTAPE 4, Dessiner en Back-to-Front :  
    pour chaque item :  
        si FourPoints -> drawWall()  
        si Sprite -> drawSprite()
```

→ Pourquoi ça marche : l'arbre BSP garantit que les murs sont déjà dans le bon ordre spatial. En insérant un sprite 'derrière le dernier mur qui le cache', on utilise la géométrie du BSP pour son placement sans z-buffer ni raycasting.

→ Limitation : fragile si un sprite se trouve exactement à cheval entre deux murs. On peut avoir certain bug graphique très spécifique où un sprite va être rendu derrière/devant le mauvais mur sur un angle précis et une configuration de murs spéciale. Mais ce sont la plupart des cas mineurs qui ne justifient pas l'implémentation d'un z-buffer coûteux complet à nos yeux.

6. Rendu final

À ce stade tout est déjà calculé et ordonné. Le Renderer se contente de dessiner dans l'ordre qu'on lui donne les FourPoints et Sprites.

```
renderWorld(orderedItems, joueur) :  
    effacer le buffer (fond noir)  
  
    pour chaque item dans orderedItems (ordre LOIN -> PROCHE) :  
  
        si FourPoints (mur) :  
            créer un Polygon Java avec les 4 coins  
            fillPolygon en gris  
            drawPolygon en blanc (contour)  
  
        si Sprite :  
            calculer l'angle entre le sprite et le joueur  
            si hors FOV -> ignorer  
            taille à l'écran = hauteur / distance  
            position X = (0.5 + angleDiff/FOV) * largeur  
            dessiner l'image centrée sur cette position  
            si le sprite a un pseudo -> afficher au-dessus  
  
    copier le buffer sur le Graphics principal (double buffering)
```

→ On utilise un BufferedImage comme buffer intermédiaire. On dessine tout dedans puis on l'affiche en une seule opération drawImage, ça optimise beaucoup la vitesse de rendu.

7. Résumé

Voilà tout ce qui se passe depuis le démarrage jusqu'à chaque frame affichée.

```
STARTUP :
  MapMur.chargerMap('fichier.txt') -> Mur[] (liste plate)
  ArbreBSP.construireBSP(MapMur)   -> NoeudBSP racine
  (l'arbre est en mémoire, il n'est jamais recalculé)

CHAQUE FRAME :
  créer FrontToBack(arbre, joueur.x, joueur.y) <- itérateur frais
  créer FilledScreen(largeur)                 <- écran vide

BOUCLE PRINCIPALE :
  mur = FrontToBack.getNextWall()
  WallCalcul.getFourPoints(mur, ...)          <- 3D -> FourPoints écran
  FilledScreen.add(FourPoints)                 <- crop + suivi occlusion
  -> morceaux visibles ajoutés dans orderedItems
  STOP si FilledScreen.isFull()

BSPParcours insère les sprites dans orderedItems
Collections.reverse(orderedItems)             <- Front-to-Back -> Back-to-Front

Renderer.renderWorld(orderedItems, joueur)
  -> fillPolygon pour chaque FourPoints
  -> drawImage pour chaque Sprite
```

La construction BSP est le coût initial qu'on amortit sur toutes les frames. Le parcours FTB + FilledScreen est l'optimisation principale (on s'arrête dès que l'écran est plein). Le reverse final + Painter's Algorithm est la technique de rendu la plus simple possible — et ça fonctionne correctement grâce à l'ordre garanti par le BSP.